

ISLAMIC UNIVERSITY OF TECHNOLOGY



MACHINE LEARNING LAB

CSE 4622

Lab 3: Linear Regression from Scratch

Author:

Ishmam Tashdeed
CSE, IUT

Contents

1	Introduction	2
2	Loading and Exploring the Data	5
2.1	Loading the Dataset	5
2.2	Quick Inspection	5
2.3	Exploratory Visualizations	6
3	Data Preprocessing	10
3.1	Removing Invalid Rows	10
3.2	Sampling for Loop-Based Training	10
3.3	Encoding Ordinal Categorical Features	10
3.4	Selecting Features and Target	12
3.5	Train / Test Split	12
3.6	Feature Scaling	12
3.7	Adding the Bias Term	13
4	Linear Regression Theory	15
4.1	The Hypothesis Function	15
4.2	The Loss Function — Mean Squared Error	15
4.3	Gradient Descent	15
4.4	Regularization	16
4.4.1	L2 Regularization (Ridge)	16
4.4.2	L1 Regularization (Lasso)	16
5	Additional Resources	17
6	Tasks	18
7	Submission Instructions	20

1 Introduction

In the previous lab we learned how to load, clean, and explore data. Now we are ready to build our first machine learning model. In this lab, we implement **Linear Regression** entirely from scratch using only Python loops and basic arithmetic — no machine learning libraries until the very end.

Building a model from scratch is the most important exercise you can do as a beginner. It forces you to understand *exactly* what is happening at every step, rather than treating a library call as a black box.

By the end of this lab you will be able to:

- Load and preprocess a tabular regression dataset.
- Write the linear hypothesis function by hand.
- Implement Mean Squared Error (MSE) loss using loops.
- Implement gradient descent using loops to learn model weights.
- Extend the loss and gradient functions to include L1 and L2 regularization.
- Compare training curves for unregularized, L1, and L2 regression.
- (Bonus) Replicate the experiment with `scikit-learn` and verify your results.

Dataset for This Lab

We will use the **Diamonds dataset**, a classic dataset containing the prices and physical attributes of 53,940 diamonds. The regression task is to predict a diamond's **price** (in USD) from features describing its weight, quality grades, and physical dimensions.

<https://www.kaggle.com/datasets/resulcaliskan/diamonds>

Download `diamonds.csv` from the link above and upload it to your Colab session using the file panel on the left.

Table 1: Description of all columns in the Diamonds dataset.

Column	Type	Description
carat	Float	Weight of the diamond (0.2 – 5.01)
cut	String (Ordinal)	Quality of the cut: Fair < Good < Very Good < Premium < Ideal
color	String (Ordinal)	Diamond color from D (best) to J (worst)
clarity	String (Ordinal)	Clarity from I1 (worst) to IF (best)
depth	Float	Total depth percentage = $2z/(x + y)$
table	Float	Width of the top facet relative to the widest point
price	Integer	Price in USD (\$326 – \$18,823) — target variable
x	Float	Length in mm
y	Float	Width in mm
z	Float	Depth in mm

Setting Up Your Environment

Open Google Colab (<https://colab.research.google.com>), create a new notebook, and name it ML_Lab3_YOUR_STUDENT_ID. Run the following cell first:

Code 1: Importing all required libraries.

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 from sklearn.model_selection import train_test_split
7 from sklearn.preprocessing import StandardScaler,
   OrdinalEncoder
8
9 # sklearn models -- used in the Bonus section only
10 from sklearn.linear_model import LinearRegression,
   Lasso, Ridge
11 from sklearn.metrics import mean_squared_error,
   r2_score

```

```
12 |
13 | %matplotlib inline
14 | sns.set_theme(style="whitegrid")
15 | print("All libraries loaded successfully!")
```

2 Loading and Exploring the Data

2.1 Loading the Dataset

Code 2: Loading the Diamonds dataset.

```
1 df = pd.read_csv("diamonds.csv")
2
3 print(f"Shape: {df.shape}")
4 df.head()
```

Expected Output

Shape: (53940, 10)

Note

The dataset has **53,940 rows** and **10 columns**. Because our loop-based gradient descent processes every sample in each epoch, training on all 53k rows would be very slow in pure Python. We will draw a random sample of **5,000 rows** for the implementation sections. The full dataset is used only for EDA visualizations.

2.2 Quick Inspection

Code 3: Inspecting data types and non-null counts.

```
1 df.info()
```

Expected Output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 53940 entries, 0 to 53939
Data columns (total 10 columns):
# Column Non-Null Count Dtype
---  ---
0 carat 53940 non-null float64
1 cut 53940 non-null object
2 color 53940 non-null object
```

```
3 clarity 53940 non-null object
4 depth 53940 non-null float64
5 table 53940 non-null float64
6 price 53940 non-null int64
7 x 53940 non-null float64
8 y 53940 non-null float64
9 z 53940 non-null float64
```

Code 4: Summary statistics and checking for invalid rows.

```
1 print(df.describe())
2
3 # Physical dimensions should never be zero for a real
  diamond
4 bad = df[(df['x'] == 0) | (df['y'] == 0) | (df['z'] ==
  0)]
5 print(f"\nRows with x=0, y=0, or z=0 (invalid): {len(
  bad)}")
```

Expected Output

```
Rows with x=0, y=0, or z=0 (invalid): 20
```

Note

The dataset has no missing values, but contains 20 rows where one or more physical dimension is zero — physically impossible for a real diamond. We will remove these during preprocessing.

2.3 Exploratory Visualizations

Code 5: Distribution of the target variable: price.

```
1 plt.figure(figsize=(11, 4))
2
3 plt.subplot(1, 2, 1)
4 plt.hist(df['price'], bins=50, color='steelblue',
  edgecolor='white')
```

```

5 plt.title("Distribution of Price (Raw)")
6 plt.xlabel("Price (USD)")
7 plt.ylabel("Count")
8
9 plt.subplot(1, 2, 2)
10 plt.hist(np.log1p(df['price']), bins=50, color='
    darkorange', edgecolor='white')
11 plt.title("Distribution of log(1 + Price)")
12 plt.xlabel("log(1 + Price)")
13 plt.ylabel("Count")
14
15 plt.suptitle("Diamond Price: Raw vs. Log-Transformed",
    fontsize=13)
16 plt.tight_layout()
17 plt.show()
18
19 print(f"Price range : ${df['price'].min()} -- ${df['
    price'].max()}")
20 print(f"Mean price : ${df['price'].mean():.0f}")
21 print(f"Median price: ${df['price'].median():.0f}")

```

Expected Output

```

Price range : $326 -- $18823
Mean price : $3933
Median price: $2401

```

Note

The price distribution is heavily **right-skewed** — most diamonds are inexpensive, but a small number are very expensive. The log-transformed distribution is much more symmetric. This skewness is important to keep in mind: our linear model will be pulled toward the expensive outliers.

Code 6: Scatter plots of all numerical features vs. price.

```

1 num_features = ['carat', 'depth', 'table', 'x', 'y', '
    z']
2

```



```

3 fig, axes = plt.subplots(2, 3, figsize=(14, 8))
4 axes = axes.flatten()
5
6 for i, col in enumerate(num_features):
7     axes[i].scatter(df[col], df['price'], alpha=0.08,
8                     color='steelblue', s=5)
9     axes[i].set_xlabel(col)
10    axes[i].set_ylabel("Price (USD)")
11    axes[i].set_title(f"{col} vs Price")
12
13 plt.suptitle("Numerical Features vs. Diamond Price",
14             fontsize=13, y=1.01)
15 plt.tight_layout()
16 plt.show()

```

Code 7: Mean price by each level of the ordinal categorical columns.

```

1 fig, axes = plt.subplots(1, 3, figsize=(15, 4))
2
3 cut_order      = ['Fair', 'Good', 'Very Good', 'Premium
4                  ', 'Ideal']
5 color_order    = ['J', 'I', 'H', 'G', 'F', 'E', 'D']
6 clarity_order  = ['I1', 'SI2', 'SI1', 'VS2', 'VS1', '
7                  VVS2', 'VVS1', 'IF']
8
9 for ax, col, order, title in zip(
10     axes,
11     ['cut', 'color', 'clarity'],
12     [cut_order, color_order, clarity_order],
13     ['Cut (worst to best)', 'Color (worst J to
14     best D)',
15     'Clarity (worst to best)']):
16     means = df.groupby(col)['price'].mean().reindex(
17         order)
18     ax.bar(order, means.values, color='steelblue',
19           edgecolor='white')
20     ax.set_title(title)
21     ax.set_xlabel(col)
22     ax.set_ylabel("Mean Price (USD)")
23     ax.tick_params(axis='x', rotation=30)

```

```

20 plt.suptitle("Mean Diamond Price by Categorical
    Feature Level",
21               fontsize=13, y=1.02)
22 plt.tight_layout()
23 plt.show()

```

Code 8: Correlation heatmap for all numerical columns.

```

1 num_cols = ['carat', 'depth', 'table', 'x', 'y', 'z',
    'price']
2 corr = df[num_cols].corr()
3
4 plt.figure(figsize=(8, 6))
5 sns.heatmap(corr, annot=True, fmt=".2f", cmap='
    coolwarm',
6               center=0, linewidths=0.5, square=True)
7 plt.title("Pearson Correlation Matrix")
8 plt.tight_layout()
9 plt.show()

```

Note

carat, x, y, and z are all very highly correlated with each other ($r > 0.95$) and with price. This is called **multicollinearity**. For linear regression, it can make weight estimates unstable. Regularization — which we implement later — helps mitigate this by shrinking the weights.

3 Data Preprocessing

3.1 Removing Invalid Rows

Code 9: Dropping rows with zero physical dimensions.

```
1 df_clean = df[(df['x'] > 0) & (df['y'] > 0) & (df['z']  
    > 0)].copy()  
2 print(f"Rows before: {len(df)}")  
3 print(f"Rows after : {len(df_clean)}")
```

Expected Output

```
Rows before: 53940  
Rows after : 53920
```

3.2 Sampling for Loop-Based Training

Running gradient descent with pure Python loops over 53,920 rows would be extremely slow. We draw a stratified random sample of **5,000 rows** to keep training times manageable while preserving the overall distribution of the data.

Code 10: Drawing a random sample of 5,000 rows.

```
1 df_sample = df_clean.sample(n=5000, random_state=42).  
    reset_index(drop=True)  
2 print(f"Sample shape: {df_sample.shape}")  
3 print(f"Price range in sample: ${df_sample['price'].  
    min()} "  
4       f"-- ${df_sample['price'].max()}")
```

Expected Output

```
Sample shape: (5000, 10)  
Price range in sample: $326 -- $18806
```

3.3 Encoding Ordinal Categorical Features

The three categorical columns — cut, color, and clarity — are all **ordinal**: they have a meaningful quality ordering. We apply **Ordinal Encoding**,

which assigns integers that preserve this ordering.

Table 2: Ordinal encoding mappings (lowest integer = worst quality).

Column	Ordering
cut	Fair (0) < Good (1) < Very Good (2) < Premium (3) < Ideal (4)
color	J (0) < I (1) < H (2) < G (3) < F (4) < E (5) < D (6)
clarity	I1 (0) < SI2 (1) < SI1 (2) < VS2 (3) < VS1 (4) < VVS2 (5) < VVS1 (6) < IF (7)

Code 11: Applying ordinal encoding to cut, color, and clarity.

```
1 cut_order      = ['Fair', 'Good', 'Very Good', 'Premium', 'Ideal']
2 color_order    = ['J', 'I', 'H', 'G', 'F', 'E', 'D']
3 clarity_order  = ['I1', 'SI2', 'SI1', 'VS2', 'VS1', 'VVS2', 'VVS1', 'IF']
4
5 enc = OrdinalEncoder(categories=[cut_order, color_order, clarity_order])
6
7 df_sample[['cut', 'color', 'clarity']] = enc.fit_transform(
8     df_sample[['cut', 'color', 'clarity']])
9
10 print(df_sample[['cut', 'color', 'clarity']].head())
```

Remember

We use Ordinal Encoding here — **not** One-Hot Encoding — because the categories have a clear quality ordering. Using One-Hot Encoding would throw away this ordering information. Using plain Label Encoding without specifying the order would assign integers arbitrarily and could imply a wrong ordering. Always match your encoding strategy to the nature of the categorical variable.

3.4 Selecting Features and Target

Code 12: Defining the feature matrix X and target vector y.

```
1 feature_cols = ['carat', 'cut', 'color', 'clarity',  
2               'depth', 'table', 'x', 'y', 'z']  
3 target_col   = 'price'  
4  
5 X_raw = df_sample[feature_cols].values.astype(float)  
6 y      = df_sample[target_col].values.astype(float)  
7  
8 print("Feature matrix shape:", X_raw.shape)    # (5000,  
9         9)  
9 print("Target vector shape :", y.shape)        #  
         (5000,)
```

3.5 Train / Test Split

We split the data **before** scaling to prevent data leakage — the scaler must be fitted only on training data.

Code 13: Splitting into 80% training and 20% test sets.

```
1 X_train, X_test, y_train, y_test = train_test_split(  
2     X_raw, y, test_size=0.2, random_state=42)  
3  
4 print(f"Training samples : {X_train.shape[0]}")  
5 print(f"Test samples      : {X_test.shape[0]}")  
6 print(f"Number of features: {X_train.shape[1]}")
```

Expected Output

```
Training samples : 4000  
Test samples    : 1000  
Number of features: 9
```

3.6 Feature Scaling

Gradient descent converges much faster when all features are on a similar scale. We apply **Standardization** (zero mean, unit variance), fitting *only* on the training set.

Code 14: Standardizing features using training statistics only.

```
1 scaler = StandardScaler()
2 X_train_scaled = scaler.fit_transform(X_train)    # fit
   + transform on train
3 X_test_scaled = scaler.transform(X_test)         #
   transform only on test
4
5 print("Training feature means (should be ~0):")
6 print(X_train_scaled.mean(axis=0).round(4))
7 print("\nTraining feature stds (should be ~1):")
8 print(X_train_scaled.std(axis=0).round(4))
```

Remember

Never call `fit_transform()` on the test set. Doing so would use test-set statistics to scale the data, which is a form of **data leakage** — your model would be indirectly “peeking” at the test set during training.

3.7 Adding the Bias Term

Linear regression requires a bias (intercept) term w_0 . The standard trick is to prepend a column of ones to X so the bias is treated exactly like any other weight.

Code 15: Prepending a bias column of ones.

```
1 def add_bias(X):
2     """Prepend a column of ones to matrix X."""
3     ones = np.ones((X.shape[0], 1))
4     return np.hstack([ones, X])
5
6 X_train_b = add_bias(X_train_scaled)
7 X_test_b  = add_bias(X_test_scaled)
8
9 print("Shape after adding bias column:", X_train_b.
   shape)
10 # (4000, 10)  --  9 features + 1 bias column
```

Note

After adding the bias column, our weight vector $\mathbf{w}$ has **10 elements**: w_0 (the bias/intercept) followed by $w_1, \dots, w_9$ (one per feature).

4 Linear Regression Theory

Before writing any code, let us establish the mathematical foundation.

4.1 The Hypothesis Function

Given an input sample $\mathbf{x} \in \mathbb{R}^{d+1}$ (with the bias 1 prepended) and a weight vector $\mathbf{w} \in \mathbb{R}^{d+1}$, the linear hypothesis predicts:

$$\hat{y} = h_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d \quad (1)$$

For the full training set $X \in \mathbb{R}^{n \times (d+1)}$, all n predictions are:

$$\hat{\mathbf{y}} = X\mathbf{w} \quad (2)$$

4.2 The Loss Function — Mean Squared Error

We measure prediction error using the **Mean Squared Error (MSE)**:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2 \quad (3)$$

where $\hat{y}^{(i)}$ is the predicted price for sample i and $y^{(i)}$ is the true price. A lower MSE means better predictions.

4.3 Gradient Descent

To minimize the loss, we repeatedly move the weights in the direction of steepest descent:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} \mathcal{L} \quad (4)$$

where $\alpha > 0$ is the **learning rate**. The partial derivative of MSE with respect to each weight w_j is:

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{2}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)} \quad (5)$$

4.4 Regularization

Regularization adds a penalty to the loss to keep the weights small and prevent overfitting.

4.4.1 L2 Regularization (Ridge)

$$\mathcal{L}_{\text{Ridge}}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2 + \lambda \sum_{j=1}^d w_j^2 \quad (6)$$

The regularization gradient contribution for weight w_j is $2\lambda w_j$. The bias w_0 is conventionally **not** regularized.

4.4.2 L1 Regularization (Lasso)

$$\mathcal{L}_{\text{Lasso}}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2 + \lambda \sum_{j=1}^d |w_j| \quad (7)$$

The gradient contribution for w_j is $\lambda \cdot \text{sign}(w_j)$, where $\text{sign}(w_j) = +1$ if $w_j > 0$, -1 if $w_j < 0$, and 0 if $w_j = 0$.

Note

Intuition for this dataset:

- **L2 (Ridge)** is particularly useful here because `carat`, `x`, `y`, and `z` are all highly correlated. Ridge shrinks all of them together rather than picking one and blowing up the others.
- **L1 (Lasso)** may drive weights of genuinely redundant features (e.g., `depth` or `table`, which have near-zero correlation with `price`) toward exactly zero.
- $\lambda \geq 0$ controls regularization strength. Because the MSE values are in the millions (squared dollars), λ must be set on a much larger scale than for unit-range targets.

5 Additional Resources

- Diamonds Dataset (Kaggle): <https://www.kaggle.com/datasets/resulcaliskan/diamonds>
- Scikit-learn Linear Models: https://scikit-learn.org/stable/modules/linear_model.html
- Pandas Documentation: <https://pandas.pydata.org/docs/>
- Matplotlib Documentation: <https://matplotlib.org/stable/contents.html>
- Andrew Ng's ML Course (Gradient Descent): <https://www.coursera.org/learn/machine-learning>

6 Tasks

Complete all tasks in your Colab notebook. Each task should be in a separate labelled cell with a Markdown header (e.g., **## Task 1**).

Task 1: Data Loading and Preprocessing

- Load `diamonds.csv` and print its shape, column types, and the first 5 rows.
- How many rows have `x`, `y`, or `z` equal to zero? Remove them and confirm the new shape.
- Draw a random sample of 5,000 rows using `random_state=42`.
- Apply Ordinal Encoding to `cut`, `color`, and `clarity` using the correct quality orderings from Table 2.
- Split into 80% train / 20% test using `random_state=42`.
- Apply `StandardScaler` (fit on train only) and prepend the bias column to both sets.
- Print the final shapes of `X_train_b` and `X_test_b`.

Task 2: Hypothesis and Loss

- Implement `predict_single(x, w)` and `predict(X, w)` using loops, where `X` is a matrix.
- Initialize a zero weight vector. Use `predict()` to make predictions on `X_train_b` and print the first 5 predicted prices alongside the true prices. What do you notice?
- Implement `mse_loss(X, y, w)` using loops. Print the initial training MSE loss (before any training).

Task 3: Gradient Descent

- Implement `compute_gradients(X, y, w)` and `gradient_descent_step(w, grads, lr)` using loops, where `lr` is the learning rate.
- Implement the full training loop that records training and test MSE every epoch.
- Train for 300 epochs with learning rate 0.05. Print the MSE every 50 epochs.
- Plot the training and test **MSE** curves on the same figure with a legend.

Task 4: Gradient Descent — With Regularization

- Implement `mse_loss_l2` and `compute_gradients_l2` for Ridge regression.
- Implement `mse_loss_l1` and `compute_gradients_l1` for Lasso. Implement the `sign()` helper yourself. If the gradient is 0, keep it 0, don't update that weight.
- Train both regularized models with $\lambda = 100$, learning rate = 0.05, for 300 epochs.
- Does regularization improve the test MSE?

Task 5: Comparison and Analysis

- Plot the **test MSE curves** for all three models on a single figure with a legend and axis labels.
- Retrain the Ridge model with $\lambda \in \{1, 10, 100, 500, 1000, 5000\}$ and plot the final test MSE vs. λ on a line plot with a logarithmic x-axis (`plt.xscale('log')`). Which λ gives the lowest test MSE?
- What happens to the weights as λ increases? Is there a sweet spot for λ ? What does a very large λ do to the model?

Task 6 (Bonus): Scikit-Learn Comparison

- Use `LinearRegression()`, `Ridge(alpha=100)`, and `Lasso(alpha=100)` from `sklearn.linear_model` to train all three models on `X_train_scaled` (without the bias column).
- Print a side-by-side comparison table of Test MSE for your implementation vs. sklearn for each method.
- Print a feature-by-feature weight comparison between your plain linear regression and sklearn's `LinearRegression`. Explain why any differences exist and how you could reduce them.

7 Submission Instructions

- Complete all tasks (Task 1–5, and optionally Task 6) in a single Google Colab notebook.
- Name your notebook: `ML_Lab3_YOUR_STUDENT_ID`
- Ensure all cells are executed and all outputs (including plots) are visible.
- Download the notebook as a **Python File** (`.py`) and submit via the course portal.

Remember

Make sure your notebook runs from top to bottom without errors before submitting. Use **Runtime** → **Restart and run all** to verify. All implementations in Tasks 1–5 must use Python loops — submissions that use `np.dot`, the `@` operator, or any sklearn model outside of Task 6 will not receive credit for those parts.